

UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Penetration Testing and Ethical Hacking

UniClass - Software Assurance Maturity Model

v1.0



DATA: 21/09/2026

Indice

1	Introduzione	2
1.1	Perchè il SAMM?	2
1.2	Cos'è UniClass?	2
1.2.1	Funzionalità di UniClass	2
1.2.2	Perchè UniClass?	2
1.2.3	Caratteristiche di UniClass	2
2	Definizione delle Fasi	3
3	Governance	3
3.1	Strategy and Metrics	3
3.2	Policy and Compliance	4
3.3	Education and Guidance	4
4	Design	4
4.1	Threat Assessment	5
4.2	Security Requirements	5
4.3	Secure Architecture	5
5	Implementation	6
5.1	Secure Build	6
5.2	Secure Deployment	6
5.3	Defect Management	7
6	Verification	7
6.1	Architecture Assessment	7
6.2	Requirements-Driven Testing	7
6.3	Security Testing	8
7	Operations	8
7.1	Incident Management	8
7.2	Environment Management	9
8	Operational Management	9
9	Note e Risoluzioni Finali	10
9.1	Compliance con OWASP SAMM	10
9.2	Tabella della Maturità	10
10	AI-Usage	11

1 Introduzione

Questo documento è definito da ciò che è stato richiesto nella Change Request per l'idea progettuale dell'esame di Penetration Testing and Ethical Hacking (PTEH). In questo documento, sarà possibile visionare solo ed esclusivamente l'utilizzo del framework OWA-SP SAMM (Software Assurance Maturity Model) [4] per le procedure determinate in principio, durante il processo manutentivo e alla fine per la verifica e validazione delle implementazioni correttive, adattive e compensative delle tecniche di sicurezza richieste.

1.1 Perché il SAMM?

Per un processo di reingegnerizzazione e manutenzione per il miglioramento dell'aspetto sicuro della Web Application di riferimento, ovvero UniClass, è sicuramente utile comprendere gli obiettivi concettuali o eventuali linee guida da considerare per lo sviluppo delle tecniche di sicurezza richieste per la validazione del progetto finale; ciò che è altrettanto importante è misurare la *postura di sicurezza* e un overview della mentalità nello sviluppo, definendo conseguentemente obiettivi, strategia, governance, design e processi metodologici per l'oggetto d'esame.

1.2 Cos'è UniClass?

UniClass è una Web Application sviluppata in ambito accademico durante il Corso di Laurea Triennale di Informatica, esposto come progetto dell'esame di Ingegneria del Software. L'interesse per il contesto applicativo, per le funzionalità (in seguito descritte) e per le tecnologie utilizzate ha portato a processi di manutenzione correttiva, adattiva e/o migliorativa nei corsi di Magistrale seguenti, migliorandone e analizzandone la sostenibilità, l'affidabilità e la struttura architetturale-comportamentale che, inizialmente, è stata severamente d'intralcio per la codifica e reificazione delle astrazioni utilizzate per rappresentare il sistema desiderato.

1.2.1 Funzionalità di UniClass

UniClass presenta funzionalità per il miglioramento della Quality of Life (QoL) degli utenti accademici presenti in Ateneo, come studenti, professori e staff. Le funzionalità sono di stampo comunicazionale (chat con utenti in piattaforma), informativo (descrizione di orari, aule, mappa, contatti) e operativo (appelli, registrazione e/o eliminazione di utenti, ...).

1.2.2 Perché UniClass?

Nonostante i diversi processi manutentivi effettuati su UniClass, l'aspetto di sicurezza non è stato il focus principale nelle Change Request precedenti, per via dell'inconsistenza degli obiettivi del corso e le tecniche che saranno citate in seguito.

1.2.3 Caratteristiche di UniClass

- JDK: 21

- **Containerization:** Docker
- **Server Build:** TomEE 10.1.3
- **Database:** PostgreSQL 17

2 Definizione delle Fasi

Come accennato in precedenza, il Framework Software Assurance Maturity Model (SAMM di OWASP) definisce quindi un modello per misurare, interpretare, analizzare e risolvere l'aspetto di sicurezza critico di un progetto all'interno di un'organizzazione o gestione basata su una governance ben definita.

Si descrive la struttura del SAMM come segue [6]:

- **Business Functions:** Categorie di attività organizzative che determinano aspetti sociotecnici dello sviluppo del sistema preso in considerazione
- **Security Practice:** Ogni macro-attività di sicurezza di una categoria viene definita come Security Practice ed ognuna ha un suo livello di maturità, da 1 a 3
- **Streams:** Ogni Security Practice definisce due diverse attività in parallelo per obiettivi differenti ma sempre coesi con la practice di riferimento

Sarà possibile, nelle sezioni successive, comprendere ogni singola Business Function e come reifieremo quella singola categoria di attività personalizzandola in base al sistema e alle pratiche di sicurezza che seguiremo.

3 Governance

La Business Function *Governance* determina la gestione generale dei processi e le attività che verranno effettuate, come la gestione della comunicazione tra i diversi team e liaison per l'analisi o sviluppo software, metodologie per l'applicazione delle pratiche, parte documentale e analitica dei processi manutentivi, etc.

3.1 Strategy and Metrics

Dal titolo autoesplicativo, si evince che la Security Practice di Strategy and Metrics definisce la parte manageriale dello sviluppo sicuro del software, definendo eventuali metriche, qualità, requisiti e acceptance testing. Secondo gli obiettivi del corso, la conoscenza del dominio applicativo e delle funzionalità presenti nell'applicazione e dato l'elenco di requisiti da rispettare nella Change Request, seguendo le linee guida OWASP e frameworks per Testing e Security Requirements con qualificazione del sistema scelto, si definisce una maturità di tipo **2** che richiede la reificazione di una roadmap unica strategica stabilire assunzioni sulla sicurezza del sistema.

Nello **Stream A** si definisce la parte creazionale e promozionale al fine di avere, come risultato, una strategia ben definita per l'applicazione, mentre nello **Stream B** dei target, KPI per la misurazione dell'efficienza.

Come è stato definito nella Change Request, la strategia utilizzata sarà l'adozione dei framework citati per l'analisi dei requisiti di sicurezza, del testing delle practice da definire con eventuale manutenzione correttiva, compensativa e/o adattiva per l'aspetto di sicurezza e successivo acceptance testing [2] [3] [5]

3.2 Policy and Compliance

La seconda Security Practice fondamentale della Business Function di Governance è sicuramente la gestione della compliance, requisiti regolamentati dalla legge esterni ed interni allineati allo scopo dell'organizzazione o gruppo di gestione del sistema specificato. Per la granularità del sistema dal punto di vista di 3rd party's libraries, sistemi comunicanti e dipendenti, requisiti di affidabilità, si evince che si richiede un grado di maturità non troppo elevato, facendo quindi parte del **1** livello, ovvero un'identificazione documentale sulla compliance organizzativa e sistemica.

Nello **Stream A** si definiscono gli standard utilizzati e si determina quindi una baseline per le polizze e regolamentazioni organizzative, mentre per lo **Stream B**, identificazione di 3rd party compliance drivers per polizze o standard *esterni al sistema e all'organizzazione*.

Data la complessità del progetto oggetto di studio relativamente bassa ad un software in fase di produzione, disponibile in tutti gli store e piattaforme con lo studio di analisi dei dati degli utenti, differenziazione tra quelli sensibili o meno e della triade CIA ben nota e analizzata secondo i livelli di granularità definiti dalla governance aziendale, si è deciso di stabilire un semplice standard e regolamentazione dell'uso dei dati che sarà parte del *Report finale operativo* riguardante le procedure di sicurezza effettuate, garantendo quindi una visione anche analitica e regolamentativa dell'operato.

3.3 Education and Guidance

In questa Business Function si parla di Training e Guida del personale facente parte dell'organizzazione per lo sviluppo sistemico del software da mettere in sicurezza e del suo aspetto manageriale, logistico, analitico e documentale. Il processo manutentivo richiesto dalla Change Request, attualmente, rivede come partecipante dell'organizzazione che gestisce il sistema solo il sottoscritto e, di conseguenza, non verrà preso in considerazione per inappropriatezza. Pertanto, il suo livello di maturità sarà **0**, essendo nullo.

4 Design

Nella Business Function di Design, non è tanto importante la parte manageriale dell'organizzazione, nonostante sia un fondamento per la gestione dei processi manutentivi presenti. Si ricorda che le Business Function sono lineari e sequenziali, di conseguenza si nota come una business function sia necessaria, (almeno per la supervisione e analisi nel maturity model OWASP SAMM), alla successiva. Dopo l'organizzazione e la gestione delle policy e aspetti deterministici per le metriche relative alla qualità dell'aspetto sicuro del software, è necessario visionare il progetto, riguardando l'alto livello di progettazione con visione di requisiti, schemi e *Threat Assessment*.

4.1 Threat Assessment

La Security Practice di Threat Assessment definisce la visione dei rischi e vulnerabilità di un sistema software in base alle sue funzionalità e l'ambiente di utilizzo. Si rivedono quindi priorità decisionali, performance di sicurezza, profilazione, problematiche e impatti. Il livello di maturità selezionato, per via del contesto e obiettivo di studio con conoscenza completa del dominio applicativo e di sviluppo è il **3**, per un miglioramento proattivo della threat coverage.

Nello **Stream A** si parla di Application Risk Profile per la comprensione e profilazione periodica del rischio con intervalli regolari per garantire efficienza nell'analisi dello stato corrente, mentre nello **Stream B** si definisce il Threat Modeling per una continua ottimizzazione e automazione nella metodologia.

Come definito anche nella CR, non si parla solo di obiettivi di CI/CD che possono automatizzare e rispondere concretamente e istantaneamente alla richiesta della comprensione dell'aspetto di sicurezza del sistema corrente, bensì anche un proattivo processo di misurazione e regolari interventi che possono essere sistemici manuali e automatici in base alle linee guida owasp [3] e **self-healing system** eventuale basato su regole [1]

4.2 Security Requirements

Per questa Security Practice, come evidenzia il titolo, si definiscono i requisiti di sicurezza per il software preso in considerazione ma c'è dell'altro: non si descrive solamente un insieme di requisiti di sicurezza che sono confinati al dominio applicativo conosciuto, bensì anche al modo in cui c'è la produzione e il rilascio del sistema, il servizio dei dati, immagazzinamento e comunicazione e, infine, librerie o dipendenze che non devono causare vulnerabilità di sicurezza determinanti negativamente per lo scopo del software (con granularità bassa, data la sua assenza nella Security Practice [4]). Data la sua importanza, l'oggetto di studio, l'obiettivo del corso e definito nella CR ed essendo area di ricerca focus del sottoscritto, si aumenta il livello di granularità e il livello di dettaglio di questa Security Practice, portandola al livello **3**.

Nello **Stream A** si parla di requisiti, definendo un framework di requisiti che l'intera organizzazione in maniera scalabile ed efficiente possa utilizzare con requisiti ad alto livello mappati ai requisiti funzionali del sistema e struttura dei primi citati, mentre per lo **Stream B** garantire un adeguato livello di sicurezza per i fornitori esterni, definendo obiettivi chiari e compliance con le regolamentazioni dell'organizzazione.

In questo caso, si parla di una definizione con aumento del livello di granularità dei requisiti di sicurezza presenti nel processo manutentivo rinnovato per l'aspetto sicuro di UniClass con struttura dei requisiti, requisiti ad alto livello per requisito funzionale e definizione di criteri d'accettazione presenti sia per l'ASVS che per il testing finale.

4.3 Secure Architecture

Questa Security Practice ridefinisce l'architettura sicura basata sull'interazione dei componenti software e le tecnologie utilizzate. Per le stesse e identiche motivazioni dei Security Requirements, si definisce un livello di granularità avanzato **3**.

Nello **Stream A** si richiede la definizione e continua analisi delle architetture utilizzate per adozione e appropriatezza con comuni design pattern usati e soluzioni di sicurezza,

mentre per lo **Stream B** c'è il *Technology Management*, ovvero imporre l'uso di standard tecnologici in tutto lo sviluppo software con standardizzazione di framework usati per tutte le applicazioni.

In questo caso si effettueranno correzioni di eventuali irregolarità nel System Design e RAD laddove presenti ed ulteriori correzioni nel *Report finale*. Inoltre, sarà possibile avere una continua analisi attraverso framework standardizzati e open riguardo le tecnologie utilizzate per l'aspetto sicuro e architetturale progettuale.

5 Implementation

Nella Business Function riguardante l'implementazione, si determina con particolare focus la totalità dei processi di sviluppo software, sulle build e ship organizzative con il minimo dei difetti possibile.

5.1 Secure Build

La Secure Build Security Practice enfatizza l'importanza delle build del software definite in maniera standardizzata, ripetibile e consistente per tutte le componenti software in ambiente sicuro, includendo dipendenze di terze parti. Anche qui, data la sua importanza, la coesione con le Security Practice della Business Function di Design, si ha un livello di granularità alto e maturità pari a **3**.

Nello **Stream A**, in particolare, si evince la possibilità di avere processi automatizzati per non consentire la creazione o produzione delle build in caso di mancata consistenza con requisiti di sicurezza impostati a posteriori per non creare un ambiente di produzione insicuro, mentre, nello **Stream B**, si definisce lo stesso ma per le dipendenze del proprio codice per problematiche eventuali di sicurezza.

Nel nostro caso, si potranno avere i requisiti dello Stream A e Stream B di maturità 3 soddisfatti attraverso un'integrazione delle pipeline CI/CD con controllo di parametri di sicurezza (eventualmente *SonarCloud/SonarQube*) e gestione delle dipendenze del codice sempre in ambito di sicurezza.

5.2 Secure Deployment

Se la prima Security Practice definisce la **CI** phase progettuale per l'integrità e la consistenza del software applicativo e determinazione di requisiti, fallback, failures, qui si ha una visione della **CD** phase, iniziando nel momento in cui l'artefatto software totale è già stato costruito e pronto per la distribuzione. Data la coesione e alto accoppiamento con gli obiettivi definiti sia nella CR che nel livello di granularità richiesto per la Practice precedente, ovvero Secure Build, si stabilisce anche qui un livello di maturità pari a **3**.

In sostanza, lo **Stream A** definisce un'automazione completa della verifica del software, mentre **Stream B** il *Secret Management*.

Grazie ai tool definiti sia poc'anzi, sia nella CR e Requisiti, sarà possibile avere le stesse metodologie risolutive del Secure Build anche per Secure Deployment (con eventuale focus esterno sui segreti).

5.3 Defect Management

La gestione dei difetti in un Software, chiamata **DM** è estremamente importante, non solo per un aspetto di sicurezza, che verrà ovviamente affrontato principalmente nel processo manutentivo attuale, bensì anche per affidabilità, disponibilità e consumo di risorse. Data l'importanza del topic e il focus di sicurezza, non sarà accettabile un livello di maturità inferiore per questa Security Practice rispetto ai livelli precedenti implementativi che costituiscono un accoppiamento sulla postura di sviluppo e build del software, di conseguenza si avrà un livello **3**.

Lo **Stream A** definisce il Defect Tracking, integrando delle SLA e tooling standardizzato (o ad hoc, rispetto ai requisiti e alla governance) per il tracking stesso, mentre lo **Stream B** gestisce le misurazioni di quest'ultimo.

Gli stessi tool saranno utili per Defect Management e saranno importanti requisiti non funzionali del nostro sistema software, aggiunti per il processo manutentivo.

6 Verification

La Business Function di Verifica non rivede banalmente la granularità nel testing per assicurarsi dell'affidabilità del software preso in considerazione, bensì anche valutazioni e/o riconsiderazioni sul processo di sviluppo, i suoi input e i suoi output.

6.1 Architecture Assessment

Questa Security Practice consiste nell'assicurarsi che sia l'architettura che l'applicazione siano in grado di soddisfare i requisiti di sicurezza e di compliance. Dato che l'architettura progettuale è definita in un esperimento in vitro, controllato e non definito per eventuali ulteriori organizzazioni o progetti partecipativi allo sviluppo, non si richiede necessariamente un livello di maturità alto, bensì medio, **2**.

Lo **Stream A** definisce l'Architecture Validation per la validazione dei meccanismi di sicurezza nelle componenti architetturali del sistema, mentre lo **Stream B** analizza l'architettura per eventuali threats e mitigazione.

L'architettura del sistema UniClass è stata revisionata più volte, soprattutto durante l'ultimo processo manutentivo per l'esame di Ingegneria del Software Tecniche Avanzate, e i risultati di quest'ultimo sono stati fortemente vantaggiosi per l'affidabilità, le risorse sprecate e manutenibilità del codice e della stessa architettura software. Pertanto, sarà comunque necessaria una visione *sicura* della stessa architettura.

6.2 Requirements-Driven Testing

Security Practice che definisce le modalità in cui ci si assicura e si valida l'implementazione manutenibile del codice presente nel sistema software, cercando di avere un focus sulla manutenibilità, sui fault e threat che si introducono ad ogni aggiornamento del sistema, indipendentemente dalla repository presente nella Configuration Management Phase. Per sicurezza non si intende semplicemente un attacco dall'interno o dall'esterno per l'appropriazione di dati sensibili (o meno), bensì anche la sicurezza *by design* di un sistema che

è resiliente e adattivo. Pertanto, il livello di maturità, dato il focus, obiettivi del corso e contesto di studio è **3**.

Lo **Stream A** determina la necessità di effettuare Regression Testing (con eventuali unit tests), mentre lo **Stream B** determina Denial of Service e Security Stress Testing.

Come anticipato all'inizio della definizione reificata della Security Practice attuale, nel processo manutentivo precedente vi è possibile visionare un livello di dettaglio nella testabilità e nella discussione dell'affidabilità del servizio ed eventuali metodi risolutivi adattivi in base alle richieste.

6.3 Security Testing

La Security Practice attuale fa leva sul fatto che, sebbene i test di sicurezza automatizzati siano rapidi e facilmente scalabili su numerose applicazioni, i test approfonditi basati su una solida conoscenza dell'applicazione e della sua logica di business sono spesso possibili solo attraverso test di sicurezza manuali, condotti da esperti, che risultano più lenti. Ciascun percorso, quindi, si basa su un unico approccio. Data la coesione con la Security Practice precedente, si ha la stessa maturità, di conseguenza **3**.

Lo **Stream A** definisce il security testing all'interno dei processi di build e deploy, mentre lo **Stream B** all'interno dello sviluppo.

Attualmente vi sono GitHub Actions per la gestione del testing in continua integrazione e deployment. L'unico punto di domanda da dover definire è il security testing, non ancora presente.

7 Operations

L'attuale Business Function definisce le attività operazionali per la triade **CIA** (Confidentiality, Integrity, Availability) del sistema preso in considerazione per tutto il suo ciclo di vita e dati associati.

7.1 Incident Management

L'attuale Security Practice revisiona la gestione degli incidenti, come breach o falle di sicurezza del sistema, necessitando di una revisione degli incidenti, operatività proattiva, revisione delle problematiche e cause con ruoli e personale predisposto alla gestione dei breach. Data la dimensione progettuale, del personale di sviluppo e campo di produzione, si definisce un livello di maturità basso, ovvero **1**.

Lo **Stream A** rivede l'Incident Detection con log per performare e arricchire le informazioni riguardanti i breach eventuali, mentre lo **Stream B** l'Incident Response con responsabilità e ruoli (il sottoscritto, banalmente).

Attualmente, è presente un sistema di Logging delle varie operazioni del database per revisione dell'affidabilità del sistema. Sarà necessario ampliare il campo del sistema di revisione proattivo per il campo di studio e gli obiettivi definiti nella CR.

7.2 Environment Management

Ci sono diversi elementi da prendere in considerazione per la sicurezza e affidabilità del sistema: molte tecnologie che fanno parte dello stack organizzativo e/o tecnologico degli attuali sistemi non sono sicure. Per questo motivo, bisogna assicurarsi un ottimo livello manutentivo di tutto lo stack del sistema preso in considerazione. Anche qui, data la granularità non particolarmente alta dello stack tecnologico attuale, si richiede un livello di maturità basso, **1**.

Lo **Stream A** definisce delle configurazioni per il sostegno dello stack attuale e con i dati a disposizione (eventualmente provenienti da altri stream), mentre lo **Stream B** definisce il patching delle componenti vulnerabili facenti parte dello stesso stack.

8 Operational Management

L'OM è estremamente importante per la gestione organizzativa e dei dati presenti nei processi di sviluppo, manutenzione e produzione. Esso definisce il mantenimento di un ottimo e sicuro stato dei dati presenti attraverso funzioni di supporto, come system provisioning, administration e decommissioning, data backup, restore, archival. Anche qui, dato il lieve grado di granularità del sistema attuale e la sua dimensione organizzativa e manageriale, si decide un livello di maturità molto basso, **1**.

Lo **Stream A** rileva le procedure protettive dei dati facenti parte in input o output nei processi di sviluppo del sistema preso in esame con semplice focus su basic security practices per la messa in sicurezza, mentre lo **Stream B** è per il System Decommissioning con eventuali migrazioni.

La risoluzione di questa practice è già definita nella gestione di Design e Implementativa tra le Business Function dell'attuale OWASP SAMM reificato per UniClass.

9 Note e Risoluzioni Finali

In questa sezione, sarà disponibile visionare l'utilità dell'OWASP SAMM, risoluzioni finali e come verranno gestite le procedure definite nel documento attuale per il processo manutentivo oggetto d'esame.

9.1 Compliance con OWASP SAMM

Grazie all'OWASP SAMM, si sono definite nuove necessità, una nuova postura di sicurezza e di sviluppo per il processo manutentivo di UniClass. Ogni sottosezione, ovvero ogni Security Practice, è composta da una definizione iniziale con motivazione sul livello di maturità richiesto, cosa è richiesto dagli stream e, come successiva e ultima nota per sottosezione, cosa ci si aspetterà dal sistema come insieme di requisiti nella CR ed eventuale focus per le procedure di sicurezza all'interno del Report Finale.

Nota bene: Usualmente si descrive un OWASP SAMM (o qualsiasi altro Maturity Model) per definire la postura *attuale* di un particolare aspetto per una determinata organizzazione. In questo caso è stata utilizzata come **definizione** di *ciò che ci si aspetta* dalla postura di sicurezza del sottoscritto nei confronti del sistema preso in esame, UniClass.

9.2 Tabella della Maturità

Tabella con maturità per ogni business function, security practice, maturità da 1 a 3, totale per business function e note.

Tabella 1: Valutazione delle Business Function di OWASP SAMM

Business Function	Security Practice	Maturity Level	Note
Governance	Strategy and Metrics	2	
	Policy and Compliance	1	
	Education and Guidance	0	Nessuna necessità per la seguente Practice.
	Totale: 3		Bassa priorità
Design	Threat Assessment	3	
	Security Requirements	3	
	Secure Architecture	3	
	Totale: 9		Alta priorità
Implementation	Secure Build	3	
	Secure Deployment	3	
	Defect Management	3	
	Totale: 9		Alta priorità
Verification	Architecture Assessment	2	
	Requirements-Driven Testing	3	
	Security Testing	3	
	Totale: 8		Alta priorità
Operations	Incident Management	1	
	Environment Management	1	
	Operational Management	1	
	Totale: 3		Bassa priorità

10 AI-Usage

È stata richiesta l'assistenza dell'IA **solo ed esclusivamente** per la formattazione della tabella. Tutto ciò che è documentato è frutto della creatività e dell'esperienza del sottoscritto.

Riferimenti bibliografici e Citazioni

- [1] Debanjan Ghosh et al. “Self-healing systems — survey and synthesis”. In: *Decision Support Systems* 42.4 (2007). Decision Support Systems in Emerging Economies, pp. 2164–2185. DOI: <https://doi.org/10.1016/j.dss.2006.06.011>. URL: <https://www.sciencedirect.com/science/article/pii/S0167923606000807>.
- [2] OWASP Foundation. *Application Security Verification Standard*. URL: <https://owasp.org/projects/asvs>.
- [3] OWASP Foundation. *Cheat Sheet Series*. URL: <https://owasp.org/projects/cheat-sheet-series>.
- [4] OWASP Foundation. *The Model*. URL: <https://owaspsamm.org/about-samm/>.
- [5] OWASP Foundation. *Web Security Testing Guide*. URL: <https://owasp.org/projects/web-security-testing-guide>.
- [6] OWASP Foundation. *What is OWASP SAMM?* URL: <https://owaspsamm.org/about/>.